

MOP — MCH Preventive Healthcare Platform

Field-officer অ্যাকাউন্ট তৈরি, mother/child নিবন্ধন ও clinical visit — সবই PostgreSQL-এ (single source of truth), SQLite শুধু offline cache

Method of Procedure · সংস্করণ 1.1 · তারিখ 06 Jul 2026 · ব্রাঞ্চ `claude/spring-boot-startup-error-d5igrb`

স্থাপত্য (Architecture) সংক্ষেপে: Backend (Spring Boot) = লেখা ও পড়ার একমাত্র কর্তৃত্ব (system of record) → **PostgreSQL**। মোবাইল অ্যাপের লোকাল **SQLite** শুধু **cache** — অনলাইনে সার্ভার থেকে রিফ্রেশ হয়, অফলাইনে পড়ার জন্য **fallback** দেয়। Risk/nutrition শ্রেণিবিন্যাস (classification) সার্ভারে হয়, তাই অ্যাপ ও পোর্টাল সবসময় একমত।

1 · সিস্টেম ডেটা-ফ্লো (Read-through + Write-through)



Field Officer App Flutter (Android) Register / Visit **Admin Portal** Supervisor view Onboard officer **SQLite cache** offline read fallback only **Spring Boot API** /api/beneficiaries /api/workers Risk / MUAC classify **PostgreSQL** users beneficiaries ① write (POST) — Postgres authoritative ② persist ③ refresh cache ④ read onboard (POST /api/workers)

পর্যায়	কী ঘটবে
① Write	অ্যাপ/পোর্টাল সরাসরি backend-এ POST করে (নিবন্ধন, visit, onboard)। PostgreSQL-ই কর্তৃত্ব।
② Persist	Backend রেকর্ড PostgreSQL-এ সেভ করে এবং risk/MUAC সার্ভার-সাইডে শ্রেণিবিন্যাস করে।
③ Refresh cache	সার্ভারের authoritative রেকর্ড দিয়ে লোকাল SQLite cache আপডেট হয়।
④ Read	UI cache থেকে পড়ে; অনলাইনে প্রতিটি লোড আগে backend থেকে রিফ্রেশ হয়, অফলাইনে cache fallback।

2 · প্রস্তুতি (একবার সেটআপ)

ধাপ	কাজ	যাচাই
0.1	PostgreSQL চালু: DB <code>preventive_healthcare</code> , user <code>zahra / zahra</code>	<code>psql</code> দিয়ে চেকা যায়
0.2	IntelliJ-এ backend Run (Rebuild করে নিন যাতে <code>application.yml</code> ও <code>db/migration</code> classpath-এ যায়)	লগে <code>jdbc:postgresql://.../preventive_healthcare</code> ও Flyway <code>V1__init</code> , port 8080

0.3	Android Studio-তে <code>mobile</code> খুলুন → <code>flutter pub get</code> → emulator-এ Run	ডিফল্ট URL <code>10.0.2.2:8080</code> backend-এ পৌঁছায়
------------	---	---

আসল ফোন (USB) হলে: `--dart-define=API_BASE_URL=http://<PC-LAN-IP>:8080` (একই WiFi)।

3 · Admin — field officer অ্যাকাউন্ট তৈরি

ধাপ	কাজ
3.1	অ্যাপে Portal/Supervisor ভিউ → লগইন <code>s.rahman / Admin-2026</code>
3.2	সাইডবার → Workers → Onboard field officer
3.3	নাম, বয়স, মোবাইল, NID, Union, Role পূরণ → Create account & generate login
3.4	Backend <code>POST /api/workers</code> → PostgreSQL-এ User + Worker সেভ → আসল username/password ফেরত ও স্ক্রিনে প্রদর্শন
3.5	Copy credentials → field officer-কে নিরাপদে দিন

"Could not reach the server" দেখালে অ্যাকাউন্ট **তৈরি হয়নি** — backend চালু করে আবার চেষ্টা করুন। এটাই Postgres-এ সেভ নিশ্চিত করে।

4 · Field Officer — mother/child নিবন্ধন ও visit

ধাপ	কাজ
4.1	Field (app) ভিউতে অংশ 3-এর credentials দিয়ে লগইন (<code>POST /api/auth/login</code> , PostgreSQL যাচাই)
4.2	New registration → ফর্ম পূরণ → Save & start first ANC visit → <code>POST /api/beneficiaries</code> → PostgreSQL-এ নতুন beneficiary
4.3	ANC visit: vitals দিন → Save → <code>POST /api/beneficiaries/{id}/anc-visit</code> → সার্ভার risk classify করে beneficiary আপডেট করে
4.4	Child assessment: MUAC/edema দিন → Save → <code>POST /api/beneficiaries/{id}/child-assessment</code> → সার্ভার SAM/MAM নির্ণয় করে আপডেট করে

5 · যাচাই — PostgreSQL-এ footprint

SQL	প্রত্যাশা
<code>SELECT username, role, name FROM users;</code>	নতুন field-officer row
<code>SELECT id, name, "union" FROM workers ORDER BY id DESC;</code>	নতুন worker row
<code>SELECT id, name, type, risk FROM beneficiaries ORDER BY id DESC;</code>	নতুন mother/child + আপডেটেড risk

সব টেবিলে এন্ট্রি থাকলে ✓ **read+write PostgreSQL** থেকে নিশ্চিত।

6 · Optimization — কেন এই ডিজাইন, ও পরবর্তী উন্নতি

যা এখন কার্যকর

- **SoT** লেখা ও পড়া দুটোই PostgreSQL-কেন্দ্রিক — একটাই সত্য উৎস।
- **Cache** SQLite শুধু cache: অনলাইনে সার্ভার থেকে রিফ্রেশ, অফলাইনে fallback।
- **Server classify** risk/MUAC সার্ভারে নির্ণীত — অ্যাপ ও পোর্টাল কখনো ভিন্ন ফল দেয় না।
- **Consistency** Admin যে credential দেখেন হুবহু তা-ই DB-তে; login ব্যর্থ হয় না।

পরবর্তী optimization (roadmap)

- **Etag / updated_at delta sync:** পুরো তালিকা নয়, শুধু পরিবর্তিত রেকর্ড টানুন — ডেটা খরচ কমে।
- **Durable offline write-queue:** অফলাইনে করা নিবন্ধন/visit payload-সহ কিউতে রেখে অনলাইনে সত্যিকারের endpoint-এ replay (শুধু ack নয়)।
- **Password hashing (BCrypt) + first-login reset** ও JWT/session টোকেন।
- **Pagination + server-side filter** (union/risk) বড় রেজিস্টারে দ্রুত লোড।
- **Optimistic UI:** সার্ভার-রেসপন্সের আগে সাময়িক প্রদর্শন, ব্যর্থ হলে rollback।
- **Connection retry/backoff** ও background sync worker।

Read/Write ডিজাইন নীতি: "Cache-aside" প্যাটার্ন — পড়ার সময় আগে সার্ভার (miss হলে cache), লেখার সময় সার্ভারে write-through করে তারপর cache আপডেট। এটি single-source-of-truth রক্ষা করে, আবার গ্রামীণ কম-সংযোগ পরিবেশে (যেমন Char Bhola) অফলাইন পড়াও সচল রাখে।

পরিবর্তিত ফাইল (এই MOP-এর কোড ভিত্তি)

- `backend/src/main/resources/application.yml` — ডিস্টেন্ট datasource → `preventive_healthcare`
- `mobile/lib/services/api_service.dart` — register + দুই visit endpoint
- `mobile/lib/services/repository.dart` — read-through + write-through
- `mobile/lib/services/local_db.dart` — cache bulk-replace
- `mobile/lib/state/app_state.dart` — write wrapper
- `mobile/lib/pages/RegisterPage.dart` — ইনপুট bind + Postgres persist
- `mobile/lib/pages/AncVisitPage.dart`, `ChildHealthPage.dart`, `OnboardModal.dart`

7 · Native PostgreSQL রান গাইড (IntelliJ + Android Studio)

লোকাল Windows-এ PostgreSQL-এর সাথে backend + Flutter অ্যাপ চালানোর সংশোধিত ধাপ।

প্রথমে ৩টি ভুল সংশোধন (এগুলো হুবহু অনুসরণ করলে **run** ব্যর্থ হবে):

প্রদর্শিত ভুল	সঠিক তথ্য (কোড অনুযায়ী)
Main class MchSurveillanceBackendApplication	সঠিক: bd.gov.mch.MchApplication
"CRITICAL STEP": @Column annotation যোগ করা	দরকার নেই — Worker / User / Beneficiary -তে @Column আগে থেকেই আছে। আবার যোগ করলে duplicate annotation → compile error ।
flutter pub run build_runner build ...	চালাবেন না — প্রজেক্টে build_runner নেই, কমান্ডটি fail করবে।

Branch খোঁজা করুন: PostgreSQL read/write, onboarding ও registration-এর সব ফিক্স আছে **claude/spring-boot-startup-error-d5igrb** ব্রাঞ্চে। আপনার ফোল্ডার যদি **...rebuild-xszyf2** হয়, তাহলে ঐ ব্রাঞ্চে ফিক্সগুলো নেই — চালানোর আগে সঠিক ব্রাঞ্চে যান/pull করুন: **git fetch origin && git checkout claude/spring-boot-startup-error-d5igrb && git pull**

পার্ট ১ — ডাটাবেজ ও ব্যাকএন্ড (IntelliJ IDEA)

ধাপ	কাজ
1.1	pgAdmin/psql-এ ডাটাবেজ তৈরি: preventive_healthcare , ইউজার zahra (পাসওয়ার্ড zahra), zahra -কে DB-র Owner বানান (Flyway-র CREATE অধিকার লাগে)।
1.2	IntelliJ-এ backend খুলুন। Maven Reload → Build ▶ Rebuild Project (যাতে application.yml ও db/migration classpath -এ কপি হয়)।
1.3	Run Configuration: Main class = bd.gov.mch.MchApplication । কোনো extra profile/env দরকার নেই — application.yml -এ ডিফল্ট datasource সেট করা আছে।
1.4	সবুজ ▶ Run। লগে Flyway V1__init সফল ও Started MchApplication দেখলে backend রেডি (port 8080)।

@Column বা entity কোড ছোঁবেন না — schema ও entity ইতিমধ্যে মিলে আছে, **ddl-auto: validate** pass করবে।

পার্ট ২ — ফ্রন্টএন্ড (Android Studio)

ধাপ	কাজ
2.1	mobile খুলুন → Terminal-এ flutter clean তারপর flutter pub get । build_runner চালাবেন না।
2.2	API URL ফাইল: mobile/lib/config/api_config.dart (constants.dart নয়)। Emulator -এর জন্য ডিফল্ট 10.0.2.2:8080 আগেই সেট — বদলানোর দরকার নেই।
2.3	আসল ফোন/Chrome হলে run-এ পাস করুন: --dart-define=API_BASE_URL=http://<PC-LAN-IP>:8080 (ফোন) বা http://localhost:8080 (Chrome web)।
2.4	Emulator চালু করে ▶ Run, অথবা Terminal-এ flutter run । অ্যাপ এখন Windows-এর PostgreSQL-এর সাথে সরাসরি যুক্ত।

MCH Preventive Healthcare Platform · MOP v1.1 · এই নথি নিরাপদভাবে সংরক্ষণ করুন। Demo credentials শুধু ডেমো বিস্তার জন্য; প্রোডাকশনে hash করা password ও token ব্যবহার করুন।